
# Izmir Democracy University

# Engineering Faculty

# Department of Electrical and Electronics Engineering

## Neural Network Final Project

## Computer Function Control with Hand Gesture


- **Academic Staff:** Dr.Öğr.Üyesi Çağlar CENGİZLER

- **Experimenters :** Salih GÜNDÜZ – 2206102021
  Deniz GÜNEŞ - 2106102050
  Mustafa Burak BAŞAR - 2206102030
  Furkan ELMAS – 2206102010
  Fatih ÇETİNKAYA - 2206102036


- **Issue Date:** 03.06.2025

- **Delivery Date:** 10.06.2025

## CONCEPTUAL INTRODUCTION

The main purpose of this project is to develop a system that can control the computer cursor and activate shortcuts according to previously taught command movements by recognizing hand movements. This artificial neural network-based system provides a wireless and natural interaction between the user and the computer by extracting information from the image.

Today, computer control is usually dependent on physical contact equipment such as keyboard, mouse or touch screen. However, studies on voice command, autonomous command with analysis of prefixed commands, command with image are ongoing today. Our project also emerged due to the desire to study in depth on new control types that are popular. Thanks to this project, the user can perform assigned commands only with hand movements without needing any wired connection or external control. This provides a great advantage especially for disabled individuals or applications that require remote interaction.

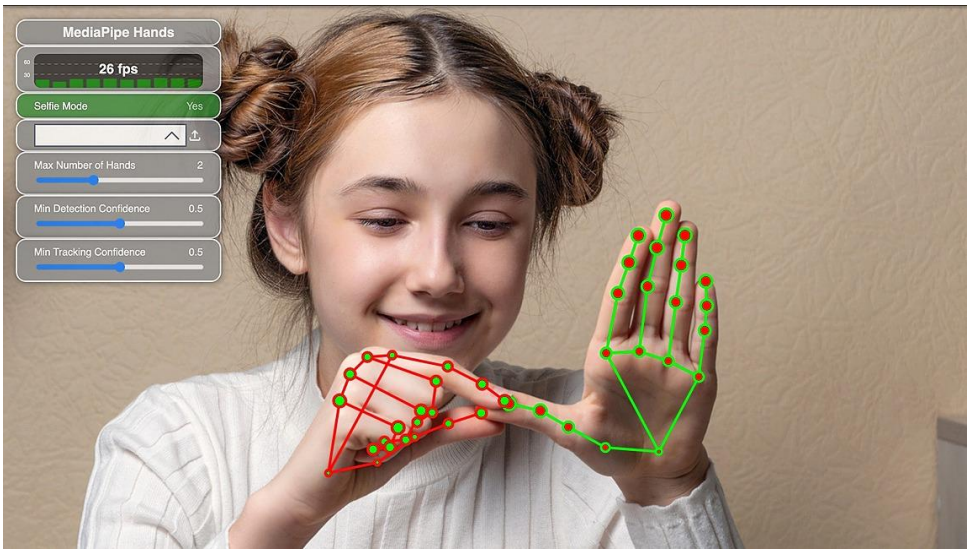
In summary, the system analyzes hand movements through images taken from the camera, identifies these movements and performs certain functions on the computer according to the defined movement.

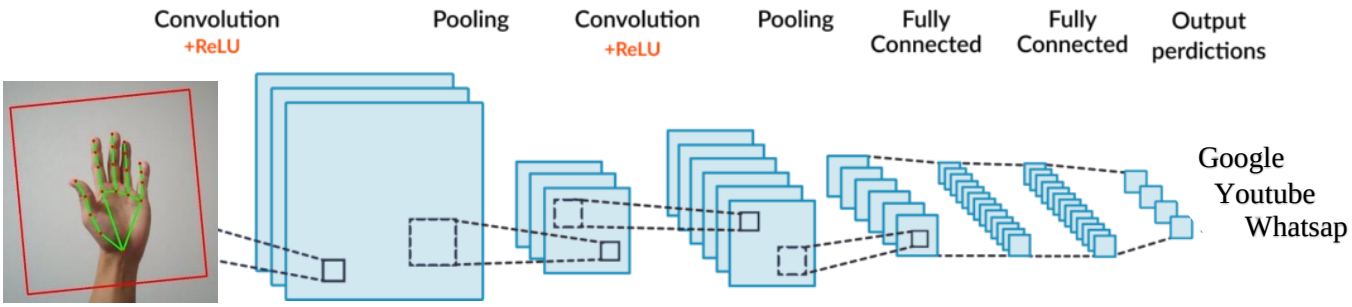
This project demonstrates how combining artificial neural networks and image processing techniques can make human-computer interaction in everyday life smoother and easier.

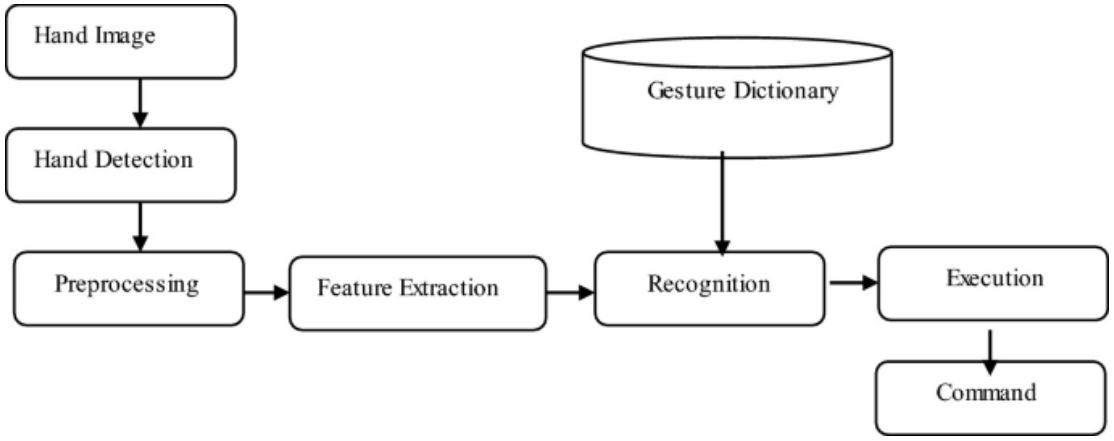
## DESIGN

### 1-) Project Phases

- **Literature Review Stage:** MediaPipe, CNN and gesture-based approaches in hand gesture recognition systems, the methods used and their working principles were examined.





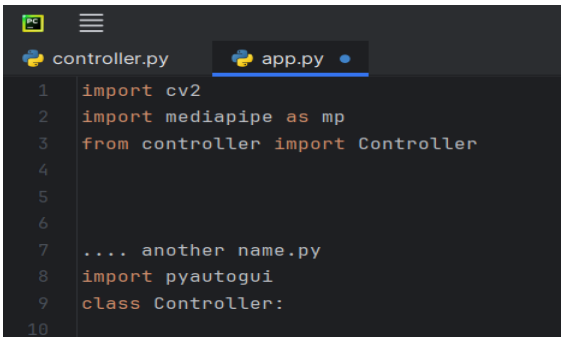

- **Roadmap Drawing Stage:** The actions to be taken were planned step by step, necessary notes were taken and then the tasks were shared.

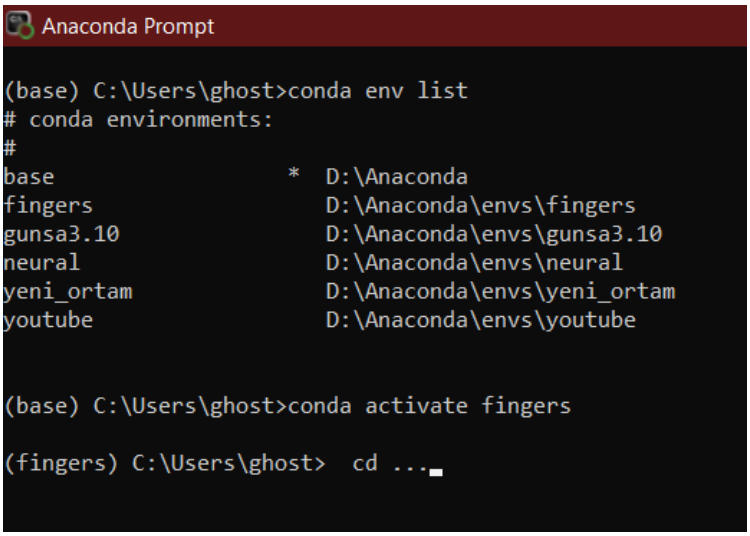
START: June 3, 2025

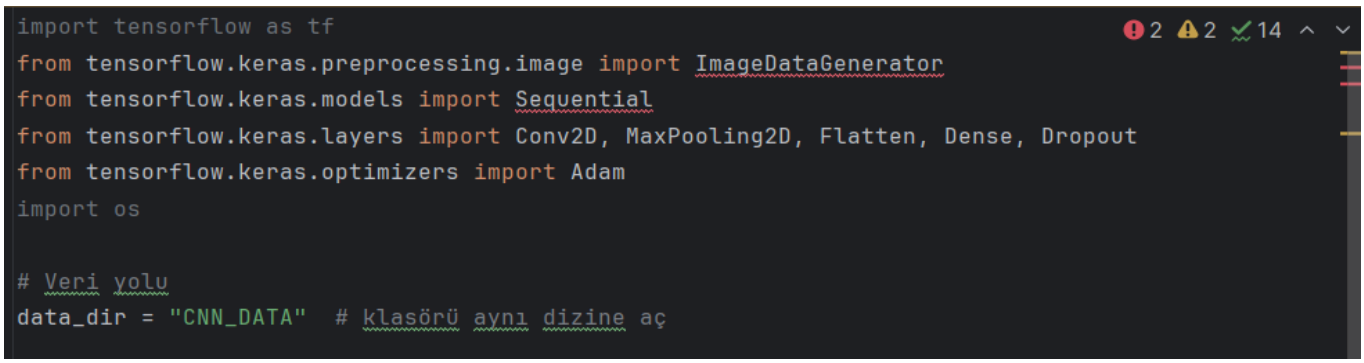
## Team Members Tasks

| | |
|---|---|
|  Salih | Project topic selection, task distribution and scheduling, researching CV2, MediaPipe, and PyAutoGUI libraries, running MediaPipe tests and preparing the demo version (part 1), building and developing the complex CNN architecture (part 2), testing and finalizing the hybrid system, analyzing execution differences between GPU and CPU-based operation, designing the system network architecture, and developing the project GUI interface. |
|  Deniz | Researching CNN architectures, examining example CNN models, designing beginner-level CNN network structure (part2 start), building and developing complex CNN model (part2), defining the project goal, What is the problem that is desired to be solved in the project and how it can be solved, documentation of resources, explanation of the libraries used, task tracking of the members in accordance with the time intervals. |
|  Mustafa | Researching CNN architectures (with a particular focus on convolution operations, pooling mechanisms, activation functions used, optimization processes, integration stages, predictions and percentage-based output confidence expressions), optimization during the CNN training process, performance testing and iterative updates, and evaluation of the system's flexibility, scalability, and future development potential. |
|  Furkan | Researching CNN architecture models, investigating the requirements for model training, collecting data for CNN model training (CNN_DATA), expanding the dataset using data augmentation methods, generating visual materials for the report, organizing and formatting the report content, explaining the working principles of the methods used in the system, and preparing the network diagrams and schematics included in the documentation. |
|  Fatih | Researching CNN architecture models, investigating the requirements for model training, collecting data for CNN model training (CNN_DATA), expanding the dataset using data augmentation methods, generating tables for the report, organizing and formatting the report structure, and researching the feasibility of running the hybrid system on a hardware platform such as Raspberry Pi with a camera module instead of a standard webcam. |

- **Preparation Phase:** The programming language is Python, the editor for writing the codes is PyCharm, and the environment for testing the code is Anaconda Prompt. In this regard, the necessary installations have been made, the necessary environments have been created, and the necessary libraries have been loaded. The gestures and functions to be used for control have been determined.

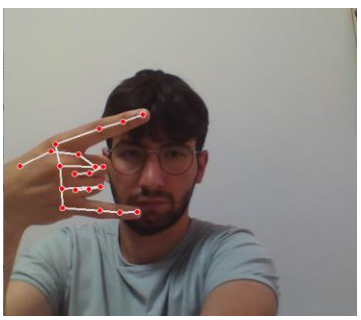



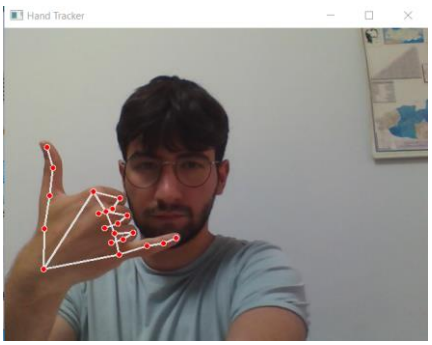

- **MediaPipe Library Testing Phase and Integration(PART-1):** The hand landmark result is based on the states of the fingers, fingertips and joints (part1). Functions are defined in the gesture reference determined for the targeted commands (controller.py part1). And the system is tested (app.py part1). Results are obtained in accordance with the targeted commands. In order to introduce the gestures and the controls represented by the gestures to the user, the necessary screenshots were taken while the system was running and stored in the folder named "gesture".

```
# ONE EXAMPLE OF CONTROL FUNCTION.

    def detect_zoomming():
        zoomming = Controller.index_finger_up and Controller.middle_finger_up and Controller.ring_finger_down and Controller.little_finger_down
        window = .05
        index_touches_middle = abs(Controller.hand_Landmarks.landmark[8].x - Controller.hand_Landmarks.landmark[12].x) <= window
        zoomming_out = zoomming and index_touches_middle
        zoomming_in = zoomming and not index_touches_middle

       # WITH TURKSH = Bu satırla zoom jesti şu şekilde tanımlanır: İşaret parmağı açık (↑) , Orta parmak açık (↑), Yüzük ve serçe parmak kapalı (↓)

        if zoomming_out:
            pyautogui.keyDown('ctrl')
            pyautogui.scroll(-50)
            pyautogui.keyUp('ctrl')
            print("Zooming Out")

       # Bu ikiparmak arasındakİ X eksenindeki mesafe ölçülür. Eğer bu mesafe 0.05'ten  küçükse, parmaklar birbirine çok yakın(yani kapanıyorlar).

        if zoomming_in:
            pyautogui.keyDown('ctrl')
            pyautogui.scroll(50)
            pyautogui.keyUp('ctrl')
            print("Zooming In")
```

- **Data Collection Stage:** The new commands and gestures to be added have been determined (3 pieces). For the data requirement to be used in training the additional CNN model (part2) in the hybrid system, the existing system (part1 = system that takes the image with CV2, performs hand landmark operation with MediaPipe and simulates cursor movements with pyautogui) was run. For each new command, landmark gestures Screen photos were taken from various angles and positions under various light intensities. 20 examples of 10 right hand gestures, 10 left, a total of 60 sample data were taken. All data were placed in different folders and they were foldered in the common "CNN_DATA" folder..

**Note:** According to the research conducted in the literature review stage, it was understood that 60 photographs for 3 classes were insufficient to train a CNN model, and every 3 new feature data were augmented within themselves with the data_augmenation method.

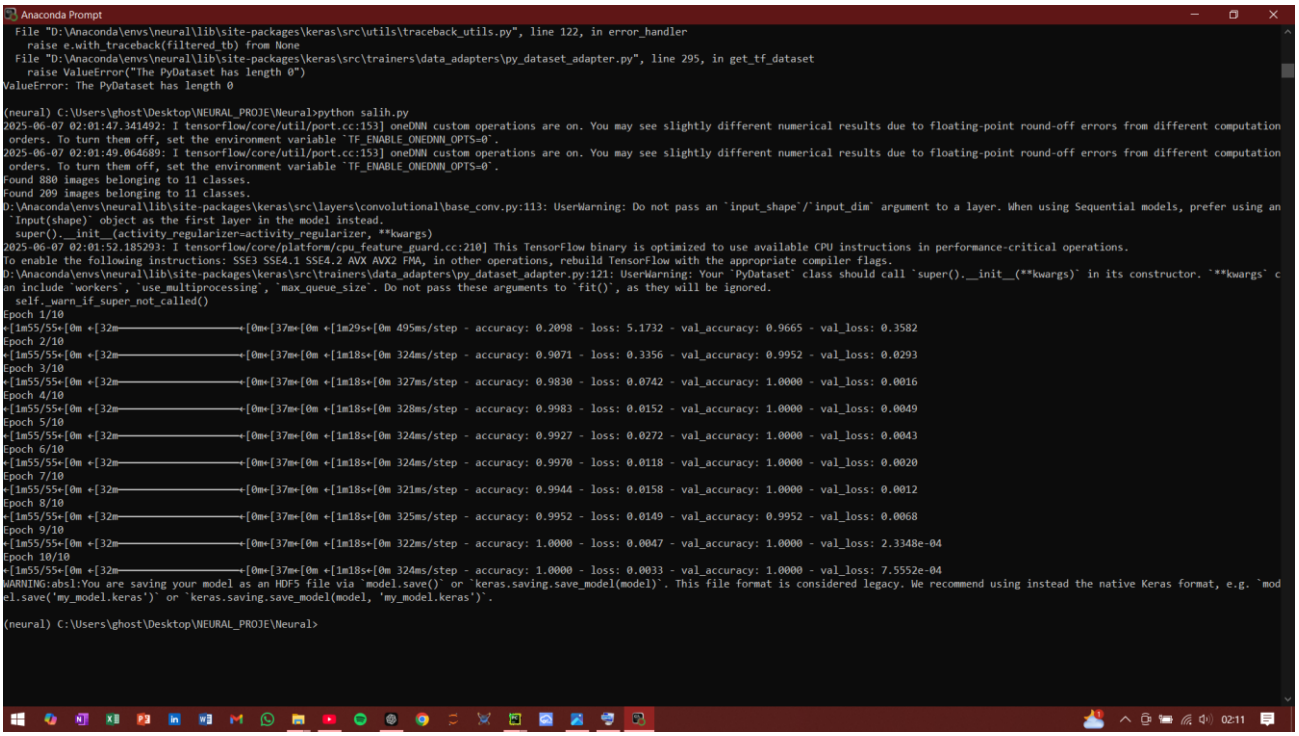


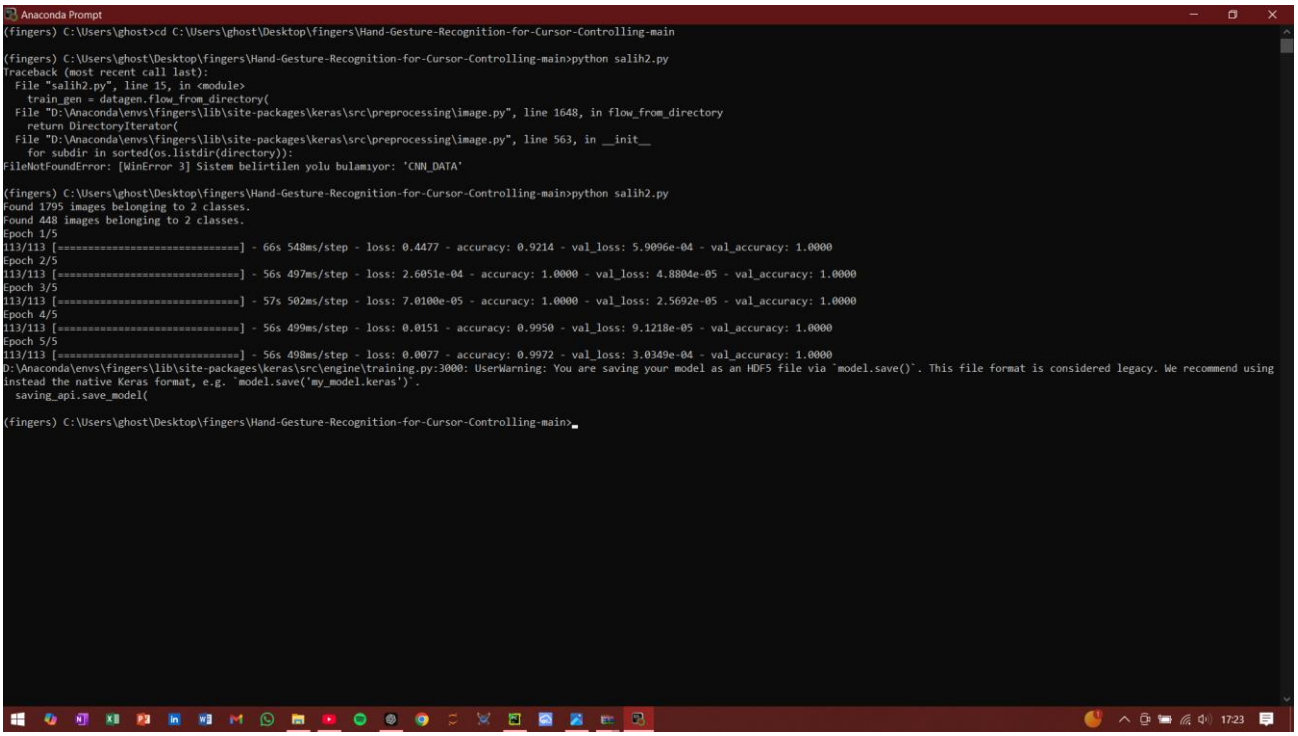



augment_gestures.py × deniz.py

```python
import os
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from PIL import Image
import numpy as np


# Veri artırma yapılandırması
datagen = ImageDataGenerator(
    rotation_range=30,
    width_shift_range=0.2,
    height_shift_range=0.2,
    zoom_range=0.2,
    shear_range=15,
    horizontal_flip=True,
    brightness_range=(0.6, 1.4),
    fill_mode='nearest'
)

# 🔁 Her görselden 10 yeni görsel üret
n_augmented = 10
count = 0
```

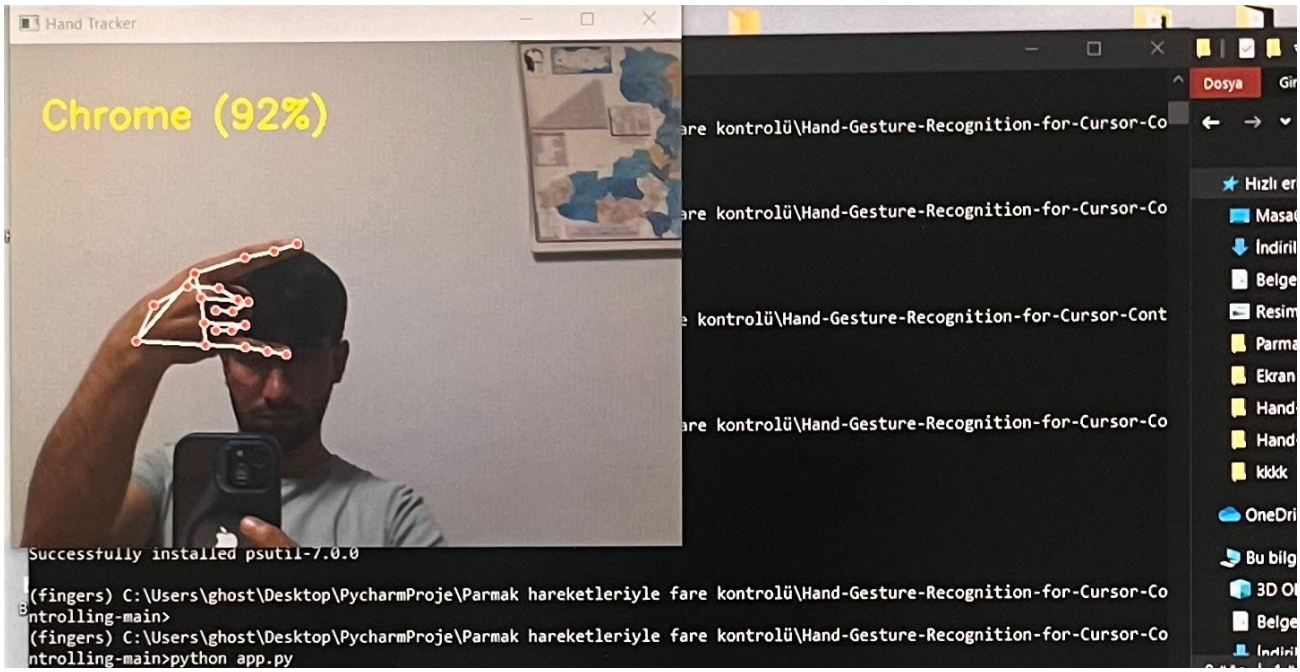
- **Hybrid CNN Model Training(PART- 2):** Additional gestures (for example: “Chrome”, “YouTube”, “Google” gestures) were defined in the previous stage. Now, a custom CNN model with the prepared CNN_DATA was trained using the TensorFlow library (controller.py part2). Then, it was integrated into the system (app.py part2).

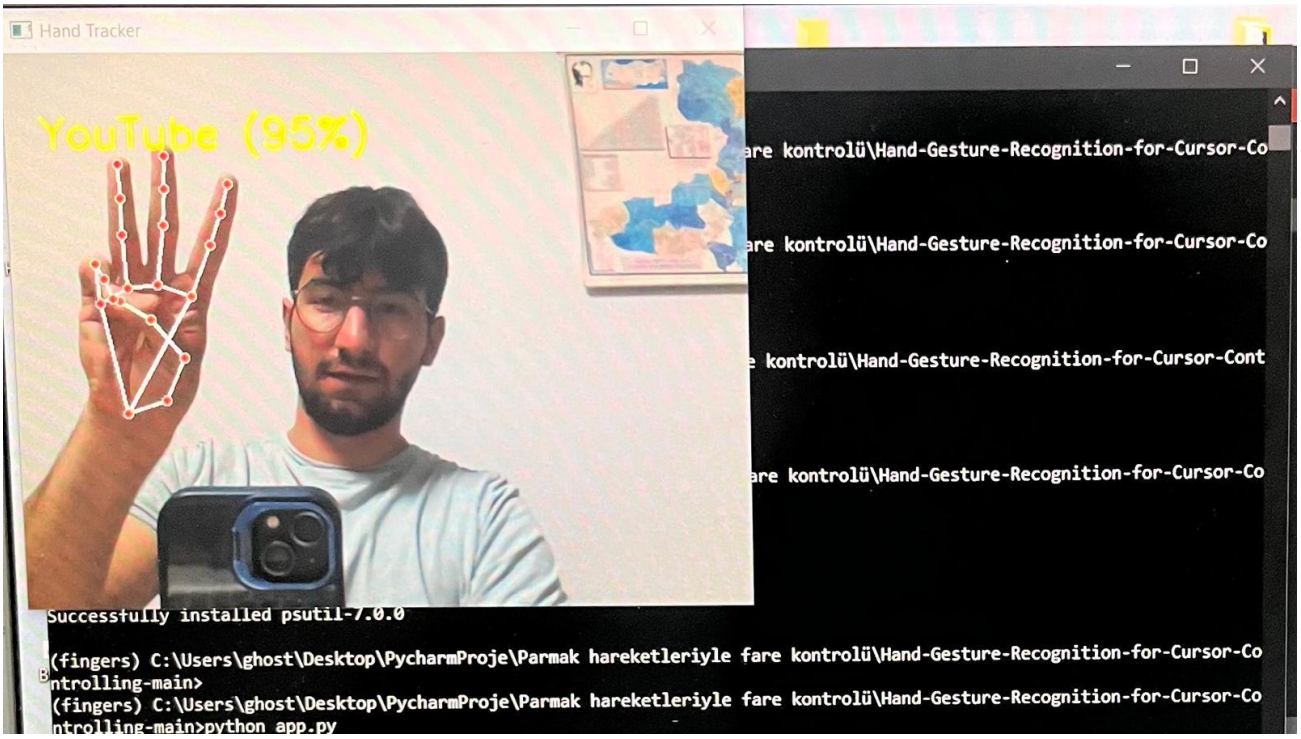


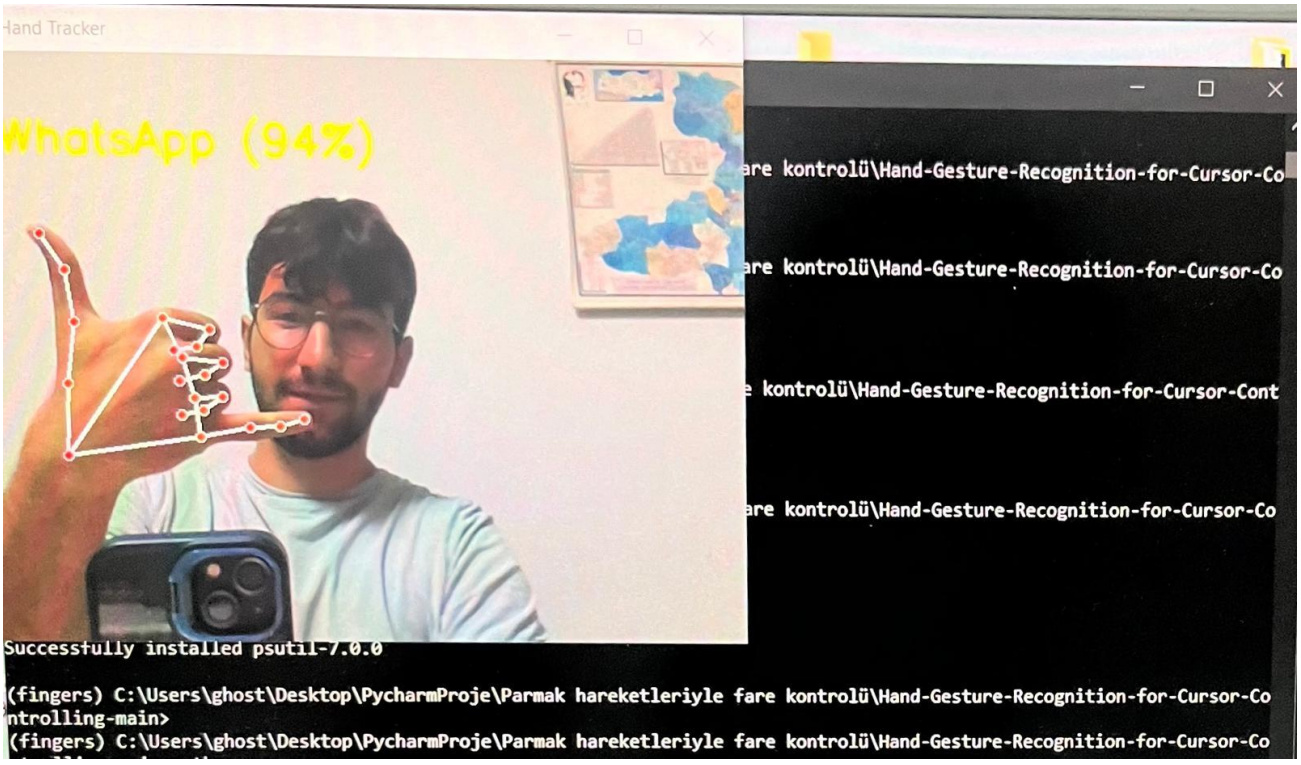



**Note:** While training the additional CNN model in the hybrid system, we experimented with different batch sizes, epochs, and other parameters to discover the most efficient values.

- **Real Time Application:** The developed system works with a webcam in real time, detects gestures and triggers the corresponding functions. The hybrid system was tested in its final form and the necessary performance values were noted. Returning to the literature stage, research and development continued and optimization was made for high accuracy and low loss function. As a result, the performance values experienced a positive development.






- **Development Process :** Solutions for the problems encountered were found and implemented.

- **GUI Design Process**: The background image was designed in Canva and buttons and functions were assigned. Then, a GUI design specific to our model was made with Tkinter.

- **Reporting and Presentation:** The entire development process is documented and experiences are reported.

## 2-) Type of Artificial Neural Network Used

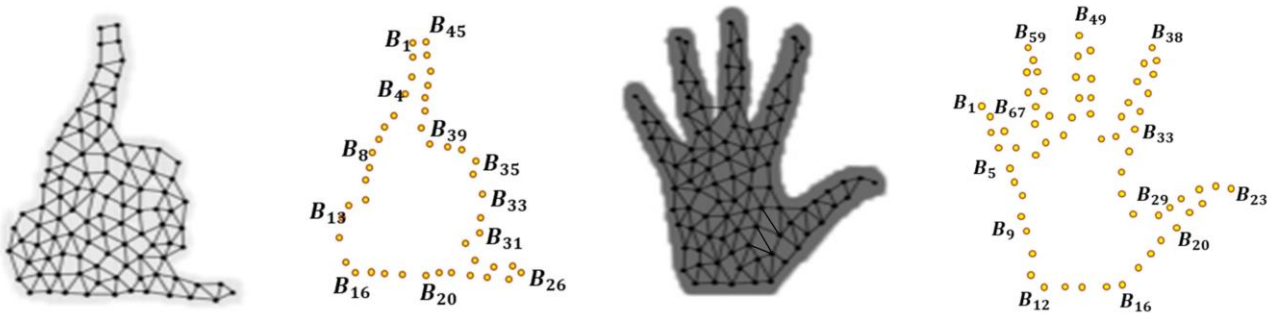
The hybrid structure of the project includes the use of two different CNN models:

### ❖ 2.1 System Network Architecture and Working Principle

CV2(OpenCV) + MediaPipe(Hand Landmark) / CNN + Rule-Based + CNN-Based + PyAutoGUI structure

➔ **MediaPipe (Embedded CNN):**

- ❖ Extracts 21 hand landmarks with a Google-trained, closed-to-user CNN model.
- ❖ This architecture works in 2 stages for detection: palm detection (palm detector) + landmark regression model.




(a) (b)

## 🧠 MediaPipe Hands Working Principle – 2 Stages

### 🔹 1. Palm Detection

In the first stage, the model detects whether there is a palm in the image.

This stage works by analyzing the general hand shape and geometry (oval structure, finger gaps, etc.).

The palm detector generates a **bounding box** for the detected region.

This region is then passed on to the second stage for detailed analysis.

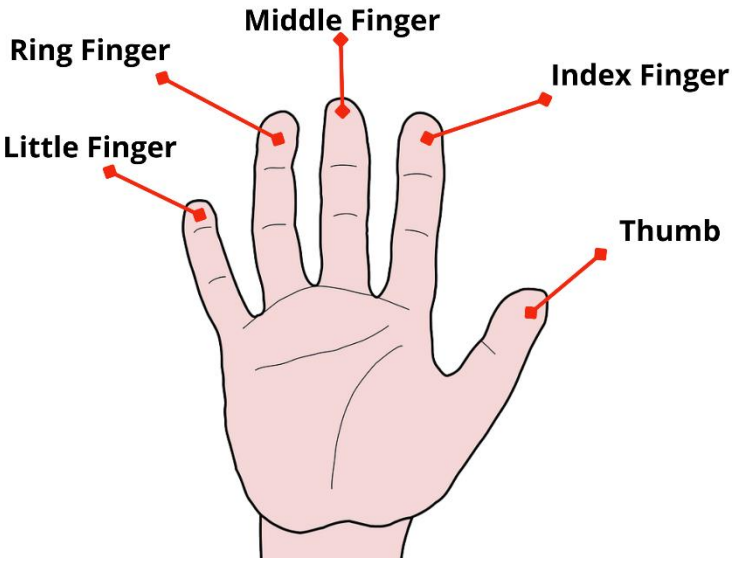
### 🔹 2. Landmark Regressor (Hand Landmark Detection)

After palm detection, the second stage identifies **21 key landmark points** within the detected hand region.

Each finger has 3–4 landmark points from fingertip to joint, resulting in 21 points for the whole hand.

These points are extracted as **3D coordinates (x, y, z)** by a CNN-based regression (position estimation) model.

❖ Output is obtained by defining functions depending on the situations.




## 🧠 Gesture Interpretation and Function Triggering

### 🔷 Gesture Interpretation

After extracting the 21 hand landmark points, the system analyzes their positions to determine the state of the hand. The main goals here are:

- Which fingers are up or down?
- Which two fingers are close to each other? (e.g., index and thumb)
- What is the relative position between fingertip and base joints?

Based on these analyses, gestures are defined. For example:

python Kopyala Düzenle

```python
if index_finger_up and middle_finger_up and ring_finger_down:
    gesture = "Zooming In"
```

### 🔷 Rule-Based Function Mapping

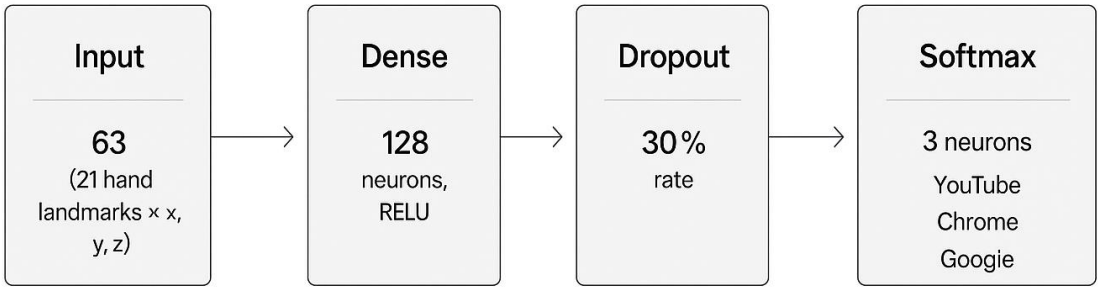
Each recognized gesture is mapped to a specific action (function) in the system. That is, **gesture → function** mapping is applied.

| Gesture | Function |
|---|---|
| All fingers up | Move mouse cursor |
| All fingers down | Initiate drag and drop |
| Thumb + pinky up | Open WhatsApp Web |
| Index + middle finger up | Zoom In |
| Only middle finger bent | Perform right click |

These actions are implemented using Python libraries such as `pyautogui`, `os.system()`, or `cv2.putText()`.

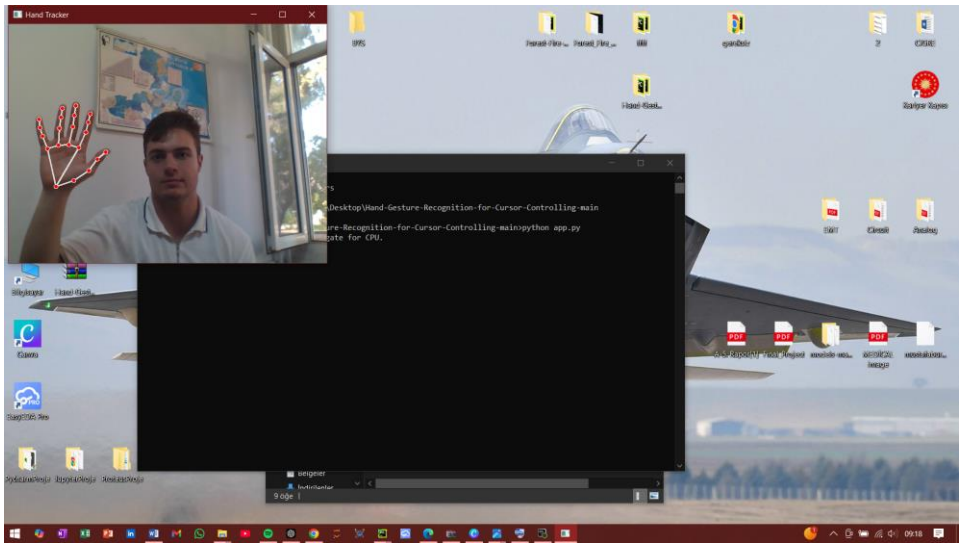
➔ **User-specific CNN (Additional Model):**

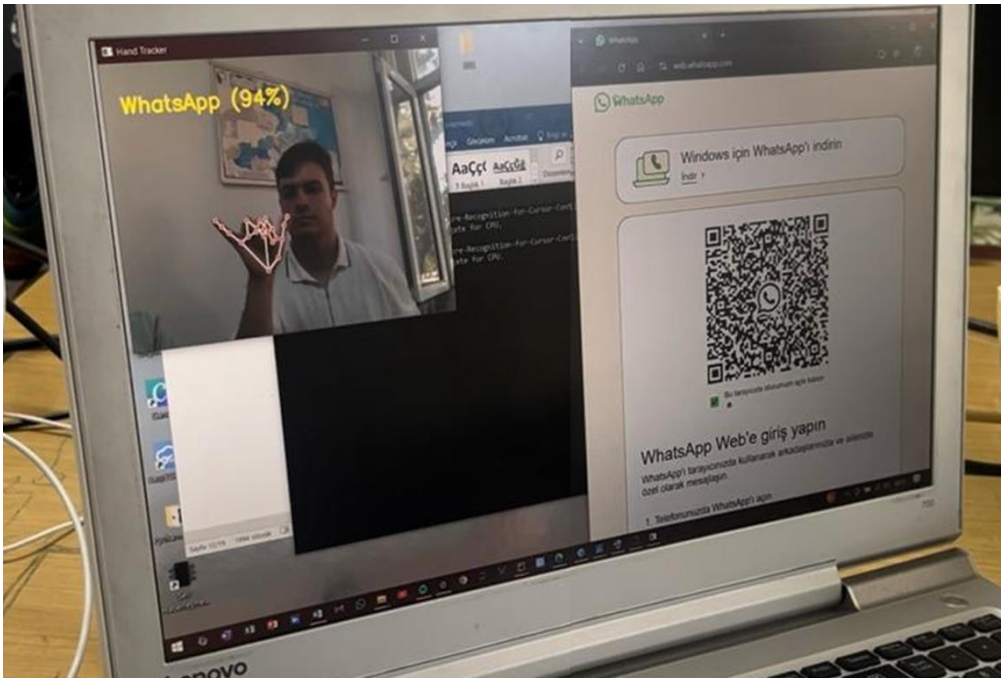
- ❖ Training data: Custom gesture examples collected from the user (e.g. YouTube sign)
- ❖ Layers:



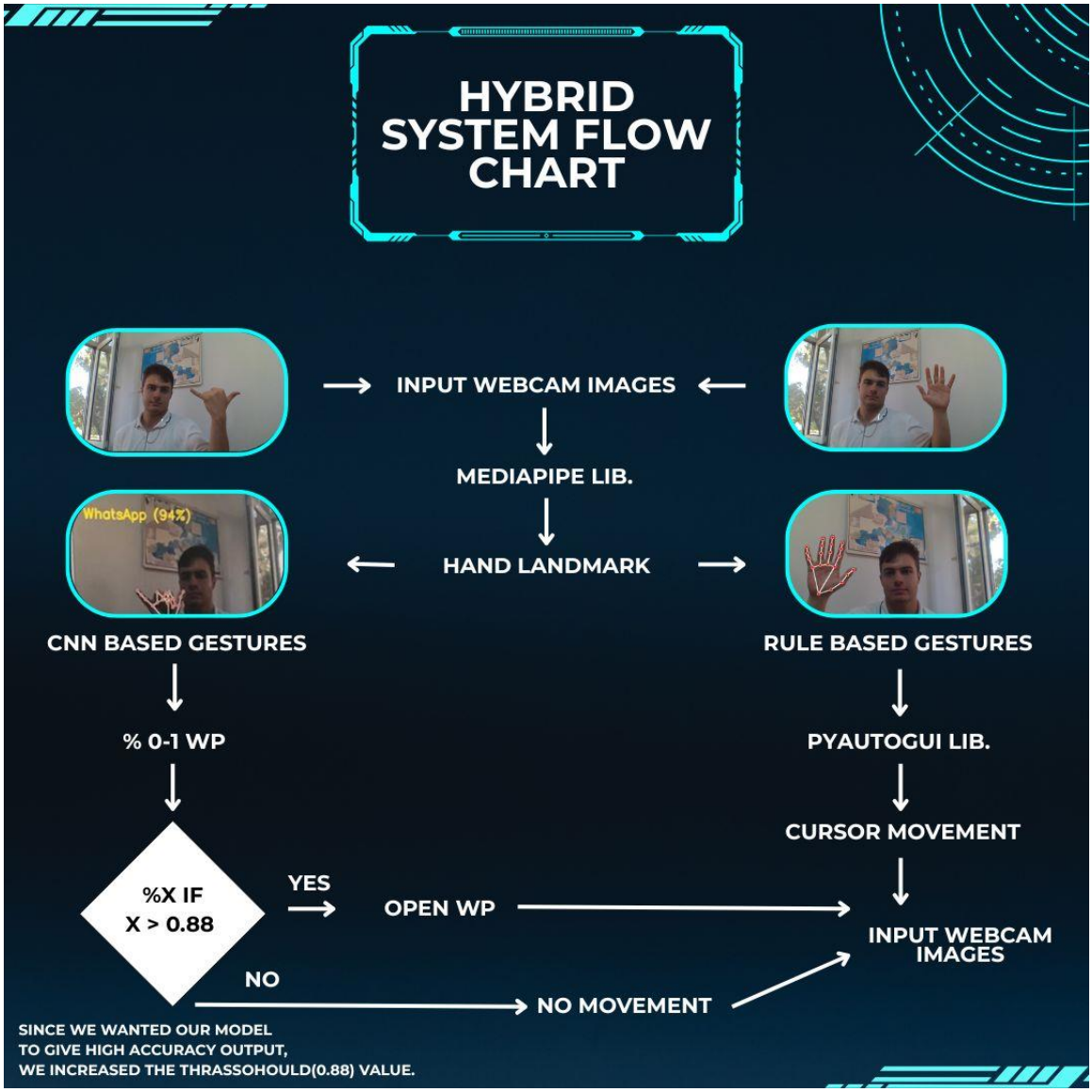

**!! The Working Principle of the Developed Hybrid System is as follows:**

1. The system continuously receives images with OpenCV via a camera connected to the computer.
2. Hand landmark data obtained with the help of MediaPipe is processed in real time.
3. Fixed defined gestures based on landmarks are directly recognized with rules (e.g. “drag”, “Move”).
4. On the other hand, original gestures can also be recognized from the user via the trained CNN model. (e.g. “youtube”, “chrome”, “whatsapp”)
5. Cursor movements, clicks or application runs are performed with the recognized gestures thanks to the pyautogui library.
6. If a gesture is shown to the system from the CNN-based trained data set, it prints the name of the current gesture and its recognition rate on the screen.






## ❖ 2.2 Data Flow Diagram



## 3-) Algorithmic Approaches

- **Rule Based Gesture Recognition:**
  Basic gestures were defined by comparing the fingers' open (up) or closed (down) states according to 21 hand landmarks obtained by MediaPipe. These definitions were created based on fixed thresholds and geometric relationships. Example: index_y < knuckle_y → finger up.

- **Learning Based Classification with CNN:**
  By collecting hand-labeled data for user's specific gestures (e.g. YouTube, Chrome, Google), a small and optimized Convolutional Neural Network (CNN) model was trained. This model took landmark vectors as input and predicted the gesture class with % confidence.

- **Process Management with psutil:**
  When a specific gesture is recognized, the application is opened, and the processes in the system background are controlled with the psutil library to ensure that this process only happens once. The same application is prevented from being opened repeatedly.

- **Instant Visual Feedback with cv2:**
  Each identified gesture was displayed as text on the screen via OpenCV (cv2). For CNN-based gestures, the screen output was provided along with the prediction percentage (for example: "YouTube (%94)").

- **Landmark Based Feature Extraction (Feature Engineering):** The obtained 3D hand points (x, y, z) were normalized and input data was created for the model. Derivative features such as distances between points, angle estimates and finger relationships also added power to the CNN model.

- **Gesture Thresholding (Thresholding):**
  Gesture repetitions and scoring thresholds were used over time to ensure that a gesture was recognized stably, thus preventing false recognitions due to single-frame noisy movements.

- **Dynamic Decision Rules for Scroll and Zoom Logic:**
  Especially for scrolling and zooming operations, not only the finger positions but also the distance between the fingertips and the direction of movement were evaluated. This algorithm worked on the principle of triggering when movement was detected instead of continuous scrolling.

- **Use of Dropout and Normalization (For CNN):**Modelin Dropout layer was used to prevent overfitting of the model during learning. Also, all landmark data were normalized in the range of 0–1 before being fed to CNN.

- **Finger Combination Control to Reduce False Positives:**
  Each click or command gesture was defined not only by the target finger movement but also by the position of the other fingers. For example: definitions such as "index in, others on" were verified by combination control, not just by relying on a single finger movement.

## 4) Live Test Stream Example

System: "Ready, you can start controlling the mouse with hand gestures."

User: opens all fingers and moves the cursor.

System: The cursor is followed on the screen.

User: Thumb and index finger are tip to tip, left click is made.

System: Left click is made.

User: Closes all fingers, dragging begins.

System: Holds down the left mouse button, cursor moves.

User: Makes a phone sign (pinky and thumb open).

System: Enters the Whatsapp application.

…

# PERFORMANCE

## 1-) Performance in Model Training

In the project, within the scope of the hybrid structure, a special artificial neural network (CNN) model trained by the user was used in addition to MediaPipe's embedded CNN model. This model was trained with previously collected hand gesture data. The data used in the training process was created with 21 landmark vectors (63 dimensions each x, y, z) taken from MediaPipe.

**→Training Stages:**

- The training data was created by considering the class balance (for example: “YouTube”, “Chrome”, “Google” gestures).
- The training set is divided into 80% and the validation set is divided into 20%.
- Training was performed with the Adam optimizer and categorical crossentropy loss function for 30–50 epochs.

**→Educational Performance (example):**

- Accuracy: **%98**
- Loss: **0.06**
- No signs of overfitting were observed (validation accuracy increased balancedly).
- Training was stabilized using techniques such as dropout and early stopping.

## 2-) Trained Model Real Test Performance

The model was tested with CNN_DATA data received from the user in real time. The test was evaluated based on recognition speed, consistency and correct command triggering with a webcam.

📌 **Evaluation Criteria:**

- **Accuracy:** Assignment of the gesture to the correct class
- **Latency:** Triggering the response without delay when the gesture is detected
- **Stability:** Consistent assignment of the same movement to the same class

**Results (with sample test data):**

| Jest | Correct Recognition Rate | Average Latency (based on CPU) | False Positive | Not |
|---|---|---|---|---|
| **YouTube** | **%95** | **~200ms** | **Low** | **Successful** |
| **Chrome** | **%92** | **~180ms** | **Middle** | **Stable** |
| **Google** | **%90** | **~190ms** | **Low** | **Stable** |

💡 Observations:

- Since the model was trained with landmark data from MediaPipe, the new 3 gesture positions trained in the additional CNN model were determined very accurately.
- In real-time tests, new gestures were identified with over 90% success.
- Latency (including camera and processing speed) is acceptable.
- In very short-term gesture changes (for example, when the finger is opened and closed quickly), errors can sometimes occur; this situation is reduced with gesture threshold filters.
- The system was tested and experimented with both GPU and CPU-based model executions in order to reduce average latency times and improve overall efficiency.

⚙ GPU vs CPU-Based Execution Comparison

| Criteria | CPU-Based Execution | GPU-Based Execution |
|---|---|---|
| Processing Structure | Sequential processing | Parallel processing (thousa-ds of cores) |
| Model Training Speed | Slower | Much faster (especially for large models) |
| Real-Time Performance | Low to medium FPS | High FPS (smother execution) |
| MediaPipe Compatibility | Fully supported (MediaPipe Python runs on CPU) | Limited (NVIDIA m Python does not use GPU) |
| Custom CNN Gesture Detection | Works, but slower inference | Faster inference and response time |
| Hardware Requirements | Runs on most systems | Requires NVIDIA GPU + CUDA |
| Setup Complexity | Easy (standard TensorFlow installation) | Complex (needs CUDA, cuDNN, proper drivers) |
| Energy Consumption | Lower | Higher (especially in laptops) |
| Overfitting/Regularization | Slower training limits experimentation | Ideal for large-scale or real-time applications |

## GUI DESIGN

## 1-) GUI Interface and Button Locations


## 2-) Button Task Descriptions

| BUTTON NAME | BUTTON FUNCTION | FUNC. DESCRIPTION |
|---|---|---|
| Start Vision Processor | Starts the gesture recognition system | Captures webcam input, uses MediaPipe to detect hand landmarks, and triggers actions based on gestures. |
| Exit Application | Closes the application | Immediately closes the GUI interface and terminates all related processes. |
| Gesture History | Displays gesture recognition history | Opens a new window showing a chronological list of all recognized gestures during runtime. |
| Personal Shortcuts | Shows the most frequently used CNN gesture | Displays the most frequently triggered gesture among user-trained commands (e.g., YouTube, Chrome, WhatsApp). |
| System Efficiency | Shows system performance metrics | Displays CPU usage, RAM usage, system version, boot time, and total number of gestures recognized. |

## CONCLUSION AND EVALUATION

In this study, **a hybrid gesture recognition system** that combines both a pre-trained CNN-based MediaPipe model and a user-trained custom CNN model, enabling computer control via hand gestures, was successfully developed.

The developed system has been expanded to include basic mouse operations (cursor movement, clicking, dragging, scrolling, zooming), as well as the ability to open specific applications (Chrome, YouTube, Google, WhatsApp Web) through user-defined gestures.

**🔍 Technical Achievements:**

- By obtaining **high-accuracy hand landmark data with MediaPipe**, a solid foundation for gesture recognition is provided.
- **Customizable command control** has become possible thanks to the **CNN model**, which can be expanded with user training.
- Thanks to the hybrid structure, the system:
    - **Performed both rule-based and learning-based recognition simultaneously,**
    - The flexibility and expandability of the system has increased.
- The developed system has achieved real-time accuracy rates of over 90% and has yielded successful results with low latency.

**🎯 Gains:**

- The project team has gained significant experience both in using pre-trained CNN architectures (MediaPipe) and in training models from scratch.

- In addition, a smarter interaction has been designed by providing application control with system-level libraries such as psutil.

- The developed system is designed to be flexible enough to be integrated with different data sources such as voice commands.

## THE FUTURE OF THE PROJECT

The developed hybrid gesture recognition system provides a flexible and customizable platform that provides computer control based on hand movements. The system can successfully perform many basic tasks in its current form and has significant potential for future development.

### 🔄 Development Potential

1. **Integration of New Data Entries:**
    - The system can be enhanced not only with hand gestures, but also with other biometric inputs such as **voice commands**, **facial expressions** or **eye movements**.
    - For voice command integration, direct command mapping can be done instead of additional model training (thanks to the flexibility provided by the hybrid structure).

2. **Creating a Comprehensive Instruction Set:**
    - The current system recognizes a limited number of gestures. By adding more user-defined gestures, the system can be made capable of managing different applications or system settings.
    - For example: Music playback, screen brightness control, window management, etc.

3. **Mobile and Embedded System Adaptations:**
    - Sistem, sadece masaüstü değil, **Android/iOS mobil cihazlara** veya **Raspberry Pi/NVIDIA Jetson** gibi gömülü platformlara uyarlanabilir.
    - Böylece temassız, düşük güçle çalışan ve taşınabilir bir kontrol sistemi elde edilmiş olur.

4. **Advanced Artificial Intelligence Integration:**
    - Instead of the current rule-based structure, behavior modeling based on past gesture data and adaptive system decisions can be implemented.
    - For example: Systems that offer frequently performed user gestures as automatic command suggestions

5. **Visual Feedback and Graphical User Interface (GUI):**
    - Visual notifications, gesture animations, and a control panel can be used in specific areas of the screen during real-time gesture recognition to enhance the user experience.
    - An accessibility-focused interface design can be considered, especially for individuals with disabilities.

### 🎯 Long-Term Goals :

- The system can behave like a **personal assistant** (similar to IronMan's JARVIS) and adapt itself according to the user's habits.
- With AI-supported recommendation systems, actions linked to gestures can be assigned dynamically.
- Storing and **synchronizing in the cloud** trained models can enable device-independent personalization.

## USED OF PYTHON LIBRARIES AND REFERENCED SOURCED

**Libraries:**

| | |
|---|---|
| **OpenCV** | **(cv2) has been used for capturing camera data, image processing, and displaying text/images on the screen.** |
| **MediaPipe** | **A library developed by Google that contains a pre-trained CNN model used for extracting hand landmark points.** |
| **PyAutoGUI** | **It has been used to perform mouse movements, clicks, and scrolling actions through software.** |
| **NumPy** | **It has been preferred for vector operations and performing mathematical computations on arrays.** |
| **psutil** | **It has been used to control background application processes and to check whether Chrome or other applications are running.** |
| **TensorFlow / Keras** | **It has been used for creating and training the CNN model that learns the user's custom gestures.** |
| **Matplotlib** | **It has been used for plotting accuracy and loss graphs during the training process.** |
| **Tkinter** | **Used to design the graphical user interface (GUI), including the application window, buttons, and background image integration.** |
| **threading** | **Allows the MediaPipe process to run in a separate thread, ensuring the GUI remains responsive during real-time gesture processing.** |
| **os** | **Used for launching applications (e.g., Chrome, YouTube) based on recognized gestures through system-level commands.** |
| **Sys** | **Used for application control and handling system-level operations like exiting the program.** |

**References:**

**Note:** The Drive link is included in the references section, and the model demo videos have been uploaded there.

➔ **https://drive.google.com/drive/folders/162cNAy2woRO3sK3kOvflY9GjH8eD_0va?usp=sharing**


1. **PyAutoGUI Resmi Belgeleri:**
   https://pyautogui.readthedocs.io/en/latest/

2. **psutil Python Kütüphanesi:**
   https://psutil.readthedocs.io/en/latest/

3. **OpenCV Python API:**
   https://docs.opencv.org/

4. **Keras & TensorFlow Belgeleri:**
   https://keras.io
   https://www.tensorflow.org

5. **Hand Gesture Recognition Paper :**
   M. Kadous, "Gesture Recognition using Time-Series Analysis", *Proceedings of the Australian Joint Conference on Artificial Intelligence*, 2002.

6. **Zhang, Y. et al. (2020).**
   *"Hand Gesture Recognition Using MediaPipe and TensorFlow."*
   arXiv preprint arXiv:2011.12002
   https://arxiv.org/abs/2011.12002

7. **Chen, J., Liang, Y., & Wang, H. (2021).**
   *"Real-time Hand Gesture Recognition Based on Deep Learning Models."*
   Proceedings of the 6th International Conference on Computer Science and Artificial Intelligence.

8. **Khan, S. et al. (2022).**
   *"A Survey of Hand Gesture Recognition Systems Using Deep Learning."*
   Computers & Electrical Engineering, Elsevier.
   https://doi.org/10.1016/j.compeleceng.2021.107006

9. **Khan, S. et al. (2022).**
*"A Survey of Hand Gesture Recognition Systems Using Deep Learning."*
Computers & Electrical Engineering, Elsevier.
https://doi.org/10.1016/j.compeleceng.2021.107006

10. **Google AI Blog (2020):**
*"MediaPipe: Cross-Platform, Customizable ML Solutions for Live and Streaming Media."*
https://ai.googleblog.com/2020/03/on-device-real-time-hand-tracking-with.html

11. **Goodfellow, I., Bengio, Y., & Courville, A. (2016).**
*"Deep Learning."* MIT Press.
– Özellikle CNN mimarisi, dropout, overfitting, eğitim süreçleri gibi temel kavramlar için referans kitaptır.
https://www.deeplearningbook.org/

12. **OpenAI Blog – GPT ve Ses Komutları Üzerine:**
*"Multimodal Interfaces and Gesture-Speech Integration."*
https://openai.com/blog/

13. **YouTube Video: Hand Gesture Control with MediaPipe + PyAutoGUI**
– Geliştirici topluluklarında pratik MediaPipe ve pyautogui entegrasyonu örnekleri
https://www.youtube.com/watch?v=9iEPzbG-xLE

14. **Sahoo, A. et al. (2023).**
*"Hybrid Gesture Control System for Touchless Human-Computer Interaction."*
IEEE Transactions on Human-Machine Systems